

Lab 6: Designing a multithreaded runtime environment for parallel computing

AMACHAT Yousra, EL HOUSSAMI Abdel Kader

November 2025

Contents

1	Introduction	3
2	Remarks :	3
3	Lab questions	3
4	Design and choices	4
4.1	Stage 1:	4
4.2	Stage 2 :	5
4.3	Stage 3 :	6
4.4	Stage 4 :	6
5	Conclusion	7

1 Introduction

In this lab, we will implement a parallel runtime environment. This will be achieved through several stages. We start off with a *stage_1* and a sequential execution. We then introduce parallelism in *stage_2*. This parallelism will then be improved throughout the following stages.

2 Remarks :

- *exec.sh* :
A bash script *exec.sh* provided in each stage. This script will compile the project using make, then it will execute all corresponding tests. Finally, **make clean** is called.
- *exec_prog_30.sh* :
A bash script that will do what *exec.sh* do, but 30 times instead of just once. This will allow us to detect concurrency problems and potential data races.
- About stage 4 :
We successfully implemented parallel execution and stealing (proved via printf's used in the steal function). This success only happens if we don't free our dynamic structures. Therefore we comment the function **delete_queues()** in the file **tasks_implem.c**. We realize that this is not a solid permanent solution, and we understand the nature of the problem.
In fact, the worker thread will keep running as long as the program is running, therefore when we call **runtime_finalize()**, the queues will be freed as the thread keeps trying to access them to get new tasks to execute. This creates a segmentation Fault.

A solution would require us to implement a termination algorithm that will make sure to stop the worker thread and killing it, thus stopping it from accessing any dynamic structure before the main thread tries to free them.

3 Lab questions

1. The provided version of the runtime uses a (single) queue to keep track of the tasks. Notice that this queue is managed with a LIFO (last in, First out) policy.
 - (a) **When is a task enqueued?** Be precise in your answer (mention the exact location in the source code).
At the end of the function **submit_task** in the file **tasks.c**, we call **dispatch_task(task_t *t)**, which is implemented in **tasks_implem.c**. This function will explicitly call **enqueue_task(tqueue, t)**;
 - (b) **When is a task dequeued?** At the end of the function **task_waitall** in the file **tasks.c**, we call **get_task_to_execute()**, which is implemented in **tasks_implem.c**. This function will explicitly call **dequeue_task(tqueue)**. Therefore, we can say that a task is dequeued before it is executed.

Exception : In stage 3 and 4, we also dequeue any parent task that should wait for its children to execute. In that case, the task will be dequeued. Then it will be enqueued when the last child finishes its execution.

2. **A given task is associated with a current status. The list of possible statuses is defined in `task_types.h` (enum `task_status`).**

- (a) **INIT status :**

Task is initialized, not submitted yet.

- (b) **READY status :**

Task is submitted, ready to be dequeued and executed.

- (c) **WAITING status :**

Task is ready, but not executing now, it is waiting for dependencies.

- (d) **RUNNING status :**

Task is being executed.

- (e) **TERMINATED status :**

Task execution is done. We terminate it.

3. **In this (non-parallel) version of the code:**

- (a) **What are the different steps performed in the `task_waitall()` function?**

The function `task_waitall`, provided in the code, will execute every single task in the queue. It will dequeue until the queue is empty, and in each iteration, execute the task. If the task is terminated, it will change its status to **TERMINATED**, or **WAITING**, if the task needs some dependencies.

- (b) **Why are all these steps performed in the `task_waitall()` function?**

The main reason for performing all these steps in the `task_waitall()` function is that the calling thread will be blocked until all tasks are executed.

For stage_1, we'll modify this function to ensure the main thread doesn't execute the tasks, but leave the execution to the children tasks.

4 Design and choices

4.1 Stage 1:

1. **When should the threads be created?**

The main thread creates the N threads in the initialization.

2. **What are the different steps executed by a worker thread?**

- Use conditional variables to wait until the common buffer is not empty.
- Acquire a lock in order to access the CS, which is, at the moment, the common task buffer.
- Dequeue the buffer and get the task to execute.
- Send a "broadcast" to tell all concerned threads that the buffer has one less element now.

- Execute the task.

- Terminate.

The worker thread will also look at the number of remaining tasks, if none is left, the thread will send a signal to the main thread telling him that he can continue the execution of his program.

3. **To which famous synchronization problem corresponds the problem to be solved for the shared queue?**

The problem corresponds to the *producer-consumer* synchronization problem. A *producer* thread will create new tasks and will want to add them to the queue, while another *consumer* thread will want to access the queue in order to execute the last added task.

4. **What should happen if the queue gets full?**

If the buffer is full, any thread trying to add new tasks to the buffer will have to wait until another thread comes and dequeue. This can be done by using conditional variables and locks.

5. **At what moment can the `task_waitall()` function return?**

The function will return when the thread executing the last task sends a signal to the main thread that called the function. Meanwhile, the caller of the function will remain blocked.

4.2 Stage 2 :

1. **Explain why the task queue resizing operation is necessary. Also briefly describe how this operation must be performed.**

The idea of having resizable queue is important to add every child task that can be created, especially that we don't know in advance the number of children that will be created by the initial threads.

2. **What does the `WAITING` task status correspond to?**

A task is in a **WAITING** status when it is waiting for its children tasks to finish their execution. This task is dependent on the execution of its children.

3. **What happens after the task routine for a task returns? To answer this question, 4 cases need to be discussed depending whether the task routine returns `TASK_COMPLETED` or `TASK_TO_BE_RESUMED` and whether the task has a parent task or not.**

- worker thread returns *TASK_COMPLETED* : The task was executed, it doesn't have any children tasks to wait for anymore. It was dequeued.
 - Task has a parent : The task is completed so `terminate_task()` is called. Thus, the *task_dependency_done* field in the parent task will be incremented then we'll check if the parent is ready to execute again. If so, the parent task's status is back to *READY*, and the parent task is enqueued again.
 - Task doesn't have a parent :
No parent task is dependent on the outcome of this task, therefore task routine ends here.

- worker thread returns *TASK_TO_BE_RESUMED* : In this case, the task's status is put to **WAITING**, the threads will continue executing the children tasks. Every time a child is executed, we will check if all of its parents' children were executed, if that's the case, we change the parent's task statuses back to **READY** and add it back to the queue so its execution is finalized.

A problem we faced in this stage, is the modification of the task's status. To solve this problem we consider any piece of code that directly changes a task's status or change it by calling **task_check_runnable()**, to be a critical section, and therefore should be protected by a lock.

4.3 Stage 3 :

In this stage we add a new structure in the file `tasks_queue.h` (Structure presented below). This new task will also be in use for the next stage. The idea is to have an array of queues, and consequently, an array of locks so that each queue has its own lock. This way enqueueing and dequeuing from different queues can be done in parallel. We also add an int field : *in* which is the index of the next queue where we'll add the next created task.

Furthermore, we had to make sure that not only the execution is done in parallel, but also the *enqueue* and *dequeue* of a task. A thread *T_1* should be allowed to add a new task to its queue *q_1* even if the thread *T_2* is currently accessing its queue *q_2*. To achieve that, we create an array of locks in the newly modified structure. And to make sure a thread can access its own lock and queue, we add an *id* field to the queue structure. The new data structure becomes as following :

Algorithm 1 Queues :

```
typedef struct tasks_queue{
    task_t** task_buffer
    unsigned int task_buf_size
    unsigned int in
    unsigned int id
} tasks_queue_t
```

Algorithm 2 Array of queues data structure :

```
typedef struct tasks_queues_array{
    tasks_queue_t** tab_queues
    pthread_mutex_t* locks_array
    unsigned int in // Next thread pour dispatch_task : Algo RR
    unsigned int out
} tasks_queues_array_t
```

4.4 Stage 4 :

To complete this stage, we add a new field to the queue structure. *steal_idx* is the index of the first element that other threads can steal. It will be initialized to -1 (case *steal_idx* = -1 is dealt with in a special if to avoid problems), then incremented every time another

thread steal the task.

The main idea here is for the worker thread to do what it was doing in stage_3, but this time, when the function `get_task_to_execute()`, returns `NULL` when the thread's corresponding queue is empty, it will go look for a task to steal in all of the other queues in a circular way. Meaning that it will check all other queues until it either finds a task to execute, or it comes back to where it was (its own queue). This is illustrated by the following algorithm :

Algorithm 3 Algo to get a task: part of the `worker_thread` function

```
while ( true ) do
    // Thread récupère une tâche
    task_t* active_tk = get_task_to_execute(id)  int j = 0  int i = 0
    while ( active_tk == NULL ) do
        if ( j == id and i != 0 ) then
            // We are back to the starting point : All queues are currently
            // empty
            break
        active_tk = steal_task(tqueue->tab_queues[(tqueue->in + j)
            %THREAD_COUNT] )
        j = (j + 1) % THREAD_COUNT
        i++
    if ( active_tk == NULL ) then
        continue
    // etc
```

5 Conclusion

In conclusion, we have implemented a multi-threaded runtime environment for parallel computing that can execute given tasks sequentially (stage_1), it can also execute all tasks in a parallel way. The parallel execution can either happen in a simplified way with a one shared queue (stage_2), or a more complicated one in which each thread has its own queue (stage_3). This version gets even more complicated, and thusly more efficient, when threads can steal tasks from one another.